

TEXT ANALYTICS - CODING MINI PROJECT (SET-D)

Name: Somil Thakur

Enrollment Number: 240212004

Programme: BS Business Analytics

Part 1 - Q1: Word2Vec

1.1 Objective

The objective of this section is to apply **Word2Vec** on a set of health and fitness sentences and understand how words can be represented as numerical vectors. The analysis covers preprocessing the text, training the model, printing the complete vocabulary, obtaining the vector representation of **diet**, finding the words most similar to **diet** and **workout**, and interpreting the results.

1.2 Approach and Word2Vec Parameters

The 20 health and fitness sentences were first stored in a Python list and saved as a CSV file. Each sentence was converted to lowercase, punctuation was replaced with spaces, tokenization was performed using NLTK's **word_tokenize**, and English stopwords were removed. The preprocessed token lists were then used to train a **Gensim Word2Vec** model with the following parameters:

Parameter	Value	Meaning	Why this value is relevant to this dataset
vector_size	50	Number of dimensions used to represent each word as a vector	The dataset has only 20 sentences and 102 vocabulary words, so a small vector of 50 numbers is enough. A very large vector would need much more text to learn useful values and would mostly capture noise.
window	3	Number of words on the left and right of a target word that are treated as its context	The sentences are short (around 6-7 words after stopwords removal), so a window of 3 already covers most of the sentence while keeping the context local.
min_count	1	Minimum number of times a word must appear to be kept in the vocabulary	Most words in this dataset appear only once. Setting it to 1 keeps every word; a higher value would remove almost the whole vocabulary (shown in Section 1.7).

Parameter	Value	Meaning	Why this value is relevant to this dataset
sg	1	Training algorithm: 1 = Skip-gram, 0 = CBOW	Skip-gram learns better vectors for rare words and works better on very small datasets like this one.
epochs	100	Number of passes the model makes over the training data	Because there are very few training examples, many passes give the model more chances to learn the relationships between words.
seed / workers	42 / 1	Random seed and number of training threads	Fixing the seed and using a single worker makes the results reproducible every time the notebook is run.

```
[9]: model = Word2Vec(
    sentences=final_sentences,
    vector_size=50,
    window=3,
    min_count=1,
    sg=1,
    epochs=100,
    seed=42,
    workers=1
)

print("Model trained!")

Model trained!
```

Figure 1: Word2Vec model configuration.

1.3 Preprocessing and Vocabulary

Before training, the sentences were cleaned and tokenized. Removing stopwords such as *a*, *is* and *the* reduced the number of unique words from **122** to **102**, so the model focuses on meaningful health and fitness words.

```
[5]: def clean_text(text):
    text = text.lower()
    for p in string.punctuation:
        text = text.replace(p, " ") # replace punctuation with a space
    return text

cleaned_sentences = []
for s in sentences:
    cleaned_sentences.append(clean_text(s))

for s in cleaned_sentences[:5]:
    print(s)
```

a balanced diet helps maintain a healthy body weight
 regular workout sessions improve overall muscle strength
 drinking enough water daily supports good digestion
 a high protein diet is common among strength athletes
 cardio workouts help improve heart and lung health

Figure 2: Converting the sentences to lowercase and removing punctuation.

```
[6]: tokenized_sentences = []
for s in cleaned_sentences:
    tokens = word_tokenize(s)
    tokenized_sentences.append(tokens)

for t in tokenized_sentences[:5]:
    print(t)
```

['a', 'balanced', 'diet', 'helps', 'maintain', 'a', 'healthy', 'body', 'weight']
 ['regular', 'workout', 'sessions', 'improve', 'overall', 'muscle', 'strength']
 ['drinking', 'enough', 'water', 'daily', 'supports', 'good', 'digestion']
 ['a', 'high', 'protein', 'diet', 'is', 'common', 'among', 'strength', 'athletes']
 ['cardio', 'workouts', 'help', 'improve', 'heart', 'and', 'lung', 'health']

Figure 3: Tokenization of the cleaned sentences using word_tokenize.

```
[7]: stop_words = set(stopwords.words('english'))

final_sentences = []
for tokens in tokenized_sentences:
    new_tokens = []
    for word in tokens:
        if word not in stop_words:
            new_tokens.append(word)
    final_sentences.append(new_tokens)

for t in final_sentences[:5]:
    print(t)

['balanced', 'diet', 'helps', 'maintain', 'healthy', 'body', 'weight']
['regular', 'workout', 'sessions', 'improve', 'overall', 'muscle', 'strength']
['drinking', 'enough', 'water', 'daily', 'supports', 'good', 'digestion']
['high', 'protein', 'diet', 'common', 'among', 'strength', 'athletes']
['cardio', 'workouts', 'help', 'improve', 'heart', 'lung', 'health']

[8]: # compare the number of unique words before and after stopword removal
words_before = set()
for tokens in tokenized_sentences:
    for w in tokens:
        words_before.add(w)

words_after = set()
for tokens in final_sentences:
    for w in tokens:
        words_after.add(w)

print("Unique words with stopwords    :", len(words_before))
print("Unique words without stopwords:", len(words_after))

Unique words with stopwords    : 122
Unique words without stopwords: 102
```

Figure 4: Stopword removal and the number of unique words before and after removal.

The dataset contains 20 sentences. After preprocessing, the Word2Vec model produced a vocabulary of **102 unique words**. Frequent words such as *diet*, *workout* and *improve* appear at the start of the vocabulary because Gensim orders words by frequency.

```
[10]: vocab = list(model.wv.index_to_key)

print("Total number of sentences:", len(sentences))
print("Vocabulary size:", len(vocab))
print()
print(vocab)

Total number of sentences: 20
Vocabulary size: 102

['diet', 'workout', 'improve', 'workouts', 'strength', 'muscle', 'balanced', 'important',
'training', 'health', 'protein', 'high', 'enough', 'body', 'schedule', 'part', 'days', 'rest',
'sufficient', 'provide', 'still', 'vegetarian', 'intense', 'prepare', 'exercises', 'warm', 'goals',
'individual', 'based', 'plan', 'design', 'often', 'nutritionists', 'levels', 'fitness', 'term',
'long', 'routines', 'consistent', 'problems', 'several', 'linked', 'sugar', 'quickly', 'calories',
'burn', 'interval', 'diseases', 'lifestyle', 'prevent', 'recommend', 'doctors', 'flexibility',
'improves', 'impact', 'low', 'yoga', 'discomfort', 'digestive', 'cause', 'change', 'sudden',
'recovery', 'sleep', 'getting', 'density', 'bone', 'build', 'grains', 'whole', 'vegetables',
'fruits', 'include', 'injury', 'risk', 'reduces', 'stretching', 'time', 'metabolism', 'slow',
'meals', 'skipping', 'lung', 'heart', 'help', 'cardio', 'athletes', 'among', 'common', 'digestion',
'good', 'supports', 'daily', 'water', 'drinking', 'overall', 'sessions', 'regular', 'weight',
'healthy', 'maintain', 'helps']
```

Figure 5: Complete Word2Vec vocabulary generated from the health and fitness dataset.

1.4 Vector Representation of diet

The trained model represents each vocabulary word as a 50-dimensional numerical vector. For the word **diet**:

- Vector shape: (50,)
- The first 10 values and the full vector are printed in the notebook output below.

```
[11]: diet_vector = model.wv['diet']

print("Shape of the vector:", diet_vector.shape)
print()
print("First 10 values:")
print(diet_vector[:10])

Shape of the vector: (50,)

First 10 values:
[-0.02545768 -0.00867327  0.00423751 -0.00316094 -0.01257511  0.02292636
 -0.02652349  0.04279565 -0.03737486 -0.00703913]

[12]: print("Full vector for 'diet':")
print(diet_vector)

Full vector for 'diet':
[-0.02545768 -0.00867327  0.00423751 -0.00316094 -0.01257511  0.02292636
 -0.02652349  0.04279565 -0.03737486 -0.00703913 -0.01558221  0.04827917
 -0.00840396  0.03589156  0.03706356  0.01478708 -0.01585216 -0.04985502
 0.00947372 -0.00540012  0.00953812  0.00133883 -0.00629369  0.03325627
 0.04958679  0.00102725 -0.02828566 -0.00254329 -0.00377864 -0.0094717
 -0.00293793 -0.02196947 -0.0461235  0.01753233  0.00294092 -0.03195698
 0.01978445 -0.04920826 -0.00080506  0.03424937 -0.01213361  0.00041616
 0.04502997 -0.03477506 -0.00981525  0.01599505  0.01090025  0.03301707
 0.02598327 -0.00745531]
```

Figure 6: 50-dimensional Word2Vec vector representation of diet.

1.5 Similarity Results

The **most_similar** function was used to find the top 5 words closest to **diet** and **workout** based on cosine similarity:

diet - similar word	Similarity	workout - similar word	Similarity
improve	0.7776	enough	0.8350
workout	0.7495	improve	0.8108
based	0.7402	diet	0.7495
enough	0.7277	workouts	0.7464
good	0.7174	muscle	0.7144

```
[13]: similar_diet = model.wv.most_similar('diet', topn=5)
```

```
print("Top 5 words similar to 'diet':")
for word, score in similar_diet:
    print(word, ":", round(score, 4))
```

```
Top 5 words similar to 'diet':
improve : 0.7776
workout : 0.7495
based : 0.7402
enough : 0.7277
good : 0.7174
```

Figure 7: Top 5 words similar to diet.

```
[14]: similar_workout = model.wv.most_similar('workout', topn=5)
```

```
print("Top 5 words similar to 'workout':")
for word, score in similar_workout:
    print(word, ":", round(score, 4))
```

```
Top 5 words similar to 'workout':
enough : 0.835
improve : 0.8108
diet : 0.7495
workouts : 0.7464
muscle : 0.7144
```

Figure 8: Top 5 words similar to workout.

```
[15]: table = pd.DataFrame({
    "diet - similar word": [w for w, s in similar_diet],
    "diet - score": [round(s, 4) for w, s in similar_diet],
    "workout - similar word": [w for w, s in similar_workout],
    "workout - score": [round(s, 4) for w, s in similar_workout]
})
table
```

```
[15]:
```

	diet - similar word	diet - score	workout - similar word	workout - score
0	improve	0.7776	enough	0.8350
1	workout	0.7495	improve	0.8108
2	based	0.7402	diet	0.7495
3	enough	0.7277	workouts	0.7464
4	good	0.7174	muscle	0.7144

Figure 9: Cosine similarity results for diet and workout in a single table.

1.6 Interpretation

The similarity scores depend on how words occur in similar contexts within this particular dataset, so only some of the results are semantically meaningful. For **workout**, the model found *workouts*, *muscle* and *improve*, which are clearly related to exercise. For **diet**, words such as *workout*, *improve* and *good* are partly sensible because diet and workout are both about health, while words like *based* and *enough* are not really related to diet.

This happens because the dataset is very small (only 20 sentences and 102 unique words) and most words appear only once, so the model does not see enough examples to learn strong meanings. Since *diet* and *workout* appear in many sentences, they also end up close to many common words. The results should therefore be read as dataset-specific patterns rather than general language relationships.

1.7 Interpretation Questions

Q1. Why is `min_count` needed? What happens if it is increased?

`min_count` tells the model to ignore words that appear fewer times than the given number. In large datasets this removes rare words and spelling mistakes that the model cannot learn well. In this small dataset most words appear only once, so increasing `min_count` shrinks the vocabulary sharply: from **102** words (`min_count = 1`) to **14** words (`min_count = 2`) and only **7** words (`min_count = 3`).

```
[16]: # small check: vocabulary size for different min_count values
for m in [1, 2, 3]:
    test_model = Word2Vec(sentences=final_sentences, vector_size=50, window=3,
                          min_count=m, sg=1, epochs=100, seed=42, workers=1)
    print("min_count =", m, "-> vocabulary size =", len(test_model.wv.index_to_key))

min_count = 1 -> vocabulary size = 102
min_count = 2 -> vocabulary size = 14
min_count = 3 -> vocabulary size = 7
```

Figure 10: Vocabulary size for different `min_count` values.

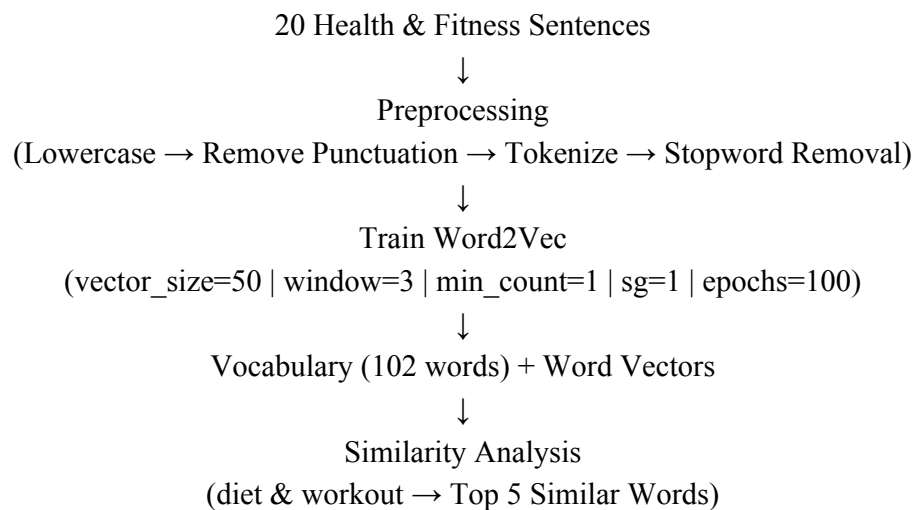
Q2. What happens when the window size is increased?

The window decides how many surrounding words are treated as context. A small window learns words that appear right next to each other (more functional or grammatical similarity), while a larger window looks at more of the sentence and learns broader, topic-level relationships. In these short sentences a large window would cover almost the whole sentence.

Q3. Why can similarity results differ after retraining?

Word2Vec starts with random vectors and uses random sampling (negative sampling) during training, and with several workers the order of training can change. Each run can therefore end with slightly different vectors and scores, and the effect is bigger on a small dataset. Setting `seed = 42` and `workers = 1` keeps the results the same every time.

1.8 Flowchart



Part 2 - Q2: Text Preprocessing

2.1 Objective

The objective of Question 2 is to preprocess a set of 12 noisy health and fitness sentences and compare different text-processing techniques: text cleaning, tokenization, stopword removal, stemming and lemmatization. The results are compared using vocabulary sizes and selected word transformations.

2.2 Approach

The 12 raw sentences contain capital letters, numbers, repeated punctuation and emoticons. They were loaded into a pandas DataFrame, saved as a CSV file, and processed through the following steps:

- **Text Cleaning:** The sentences were converted to lowercase, everything except letters and spaces (punctuation, numbers and special characters) was removed with a regular expression, and multiple spaces were collapsed into one. This keeps the text consistent so that variations like *DIET* and *diet* are not treated as different tokens.
- **Tokenization:** The cleaned sentences were split into words using two methods: NLTK's `word_tokenize` and `RegexpTokenizer` with the pattern `[a-zA-Z]+`. Both were also run on the raw text to show the real difference between them.
- **Stopword Removal:** Common English stopwords were removed using NLTK's stopword list, and the token counts before and after removal were recorded for every sentence.
- **Stemming:** The remaining words were reduced with the **Porter Stemmer** and the **Snowball Stemmer**. Both cut word endings using fixed rules and do not check a dictionary, so they can produce forms such as *daily* → *daili*.
- **Lemmatization:** The **WordNet Lemmatizer** was applied by first treating each word as a verb and, if nothing changed, as a noun. This returns real dictionary forms, for example *improves* → *improve* and *calories* → *calorie*.
- **Comparison:** Porter, Snowball and WordNet outputs were compared word by word, and vocabulary sizes were calculated after each stage to see the effect of each technique.

```
[2]: raw_sentences = [
    "A balanced DIET helps maintain healthy body weight!!",
    "regular workout sessions improve muscle strength... love it :)",
    "drinking 8 glasses of water daily supports digestion!!",
    "skipping meals can SLOW down your metabolism...",
    "stretching before a workout reduces injury risk!! important",
    "strength training builds muscle & improves bone density",
    "getting 7-8 hrs of sleep is important for recovery :)",
    "yoga is a LOW-impact workout, improves flexibility!!",
    "a high-sugar diet is linked to health problems...",
    "interval training burns calories QUICKLY!! great results",
    "nutritionists design diet plans based on individual goals",
    "rest days are an important part of any workout schedule!!"
]

df = pd.DataFrame({"text": raw_sentences})

# save the dataset as dataset.csv
df.to_csv("dataset.csv", index=False)

df
```

[2]:

	text
0	A balanced DIET helps maintain healthy body we...
1	regular workout sessions improve muscle streng...
2	drinking 8 glasses of water daily supports dig...
3	skipping meals can SLOW down your metabolism...
4	stretching before a workout reduces injury ris...
5	strength training builds muscle & improves bon...
6	getting 7-8 hrs of sleep is important for reco...
7	yoga is a LOW-impact workout, improves flexibi...
8	a high-sugar diet is linked to health problems...
9	interval training burns calories QUICKLY!! gre...
10	nutritionists design diet plans based on indiv...
11	rest days are an important part of any workout...

Figure 11: Loading the 12 raw sentences into a DataFrame and saving dataset.csv.

2.3 Text Cleaning and Tokenization

The cleaning function lowercases the text, keeps only letters and spaces, and removes extra spaces. The raw and cleaned sentences are shown side by side below; for example *"A balanced DIET helps maintain healthy body weight!!"* becomes *"a balanced diet helps maintain healthy body weight"*.

```
[3]: def clean_text(text):
      text = text.lower()           # 1. lowercase
      text = re.sub(r'^a-z\s', ' ', text) # 2. keep only letters and spaces
      text = re.sub(r'\s+', ' ', text)    # 3. collapse extra spaces
      text = text.strip()
      return text

      df["clean_text"] = df["text"].apply(clean_text)

[4]: pd.set_option('display.max_colwidth', None)
      pd.set_option('display.max_rows', 100)
      df[["text", "clean_text"]]

[4]:
```

	text	clean_text
0	A balanced DIET helps maintain healthy body weight!!	a balanced diet helps maintain healthy body weight
1	regular workout sessions improve muscle strength... love it :)	regular workout sessions improve muscle strength love it
2	drinking 8 glasses of water daily supports digestion!!	drinking glasses of water daily supports digestion
3	skipping meals can SLOW down your metabolism...	skipping meals can slow down your metabolism
4	stretching before a workout reduces injury risk!! important	stretching before a workout reduces injury risk important
5	strength training builds muscle & improves bone density	strength training builds muscle improves bone density
6	getting 7-8 hrs of sleep is important for recovery :)	getting hrs of sleep is important for recovery
7	yoga is a LOW-impact workout, improves flexibility!!	yoga is a low impact workout improves flexibility
8	a high-sugar diet is linked to health problems...	a high sugar diet is linked to health problems
9	interval training burns calories QUICKLY!! great results	interval training burns calories quickly great results
10	nutritionists design diet plans based on individual goals	nutritionists design diet plans based on individual goals
11	rest days are an important part of any workout schedule!!	rest days are an important part of any workout schedule

Figure 12: Cleaning function and the raw vs cleaned sentences.

The cleaned sentences were then tokenized with `word_tokenize` and `RegexpTokenizer`. On the cleaned text both methods produced exactly the same tokens because no punctuation or numbers were left. On the raw text, however, `word_tokenize` keeps punctuation (!, ..., :,) and numbers (8) as separate tokens, while `RegexpTokenizer` keeps only alphabetic words.

```
[5]: # (a) NLTK word_tokenize
df["word_tokens"] = df["clean_text"].apply(word_tokenize)

# (b) RegexpTokenizer that keeps only alphabetic words
regexp_tokenizer = RegexpTokenizer(r'[a-zA-Z]+')
df["regexp_tokens"] = df["clean_text"].apply(regexp_tokenizer.tokenize)

for i in range(3):
    print("Sentence", i + 1, ":", df["clean_text"][i])
    print("word_tokenize      :", df["word_tokens"][i])
    print("RegexpTokenizer      :", df["regexp_tokens"][i])
    print()

Sentence 1 : a balanced diet helps maintain healthy body weight
word_tokenize      : ['a', 'balanced', 'diet', 'helps', 'maintain', 'healthy', 'body', 'weight']
RegexpTokenizer      : ['a', 'balanced', 'diet', 'helps', 'maintain', 'healthy', 'body', 'weight']

Sentence 2 : regular workout sessions improve muscle strength love it
word_tokenize      : ['regular', 'workout', 'sessions', 'improve', 'muscle', 'strength', 'love', 'it']
RegexpTokenizer      : ['regular', 'workout', 'sessions', 'improve', 'muscle', 'strength', 'love', 'it']

Sentence 3 : drinking glasses of water daily supports digestion
word_tokenize      : ['drinking', 'glasses', 'of', 'water', 'daily', 'supports', 'digestion']
RegexpTokenizer      : ['drinking', 'glasses', 'of', 'water', 'daily', 'supports', 'digestion']
```

Figure 13: Tokenization of the cleaned text using `word_tokenize` and `RegexpTokenizer`.

```
[6]: for i in range(3):
    print("Raw sentence", i + 1, ":", df["text"][i])
    print("word_tokenize      :", word_tokenize(df["text"][i]))
    print("RegexpTokenizer      :", regexp_tokenizer.tokenize(df["text"][i]))
    print()

Raw sentence 1 : A balanced DIET helps maintain healthy body weight!!
word_tokenize      : ['A', 'balanced', 'DIET', 'helps', 'maintain', 'healthy', 'body', 'weight', '!', '!']
RegexpTokenizer      : ['A', 'balanced', 'DIET', 'helps', 'maintain', 'healthy', 'body', 'weight']

Raw sentence 2 : regular workout sessions improve muscle strength... love it :)
word_tokenize      : ['regular', 'workout', 'sessions', 'improve', 'muscle', 'strength', '...', 'love', 'it', ':', ')']
RegexpTokenizer      : ['regular', 'workout', 'sessions', 'improve', 'muscle', 'strength', 'love', 'it']

Raw sentence 3 : drinking 8 glasses of water daily supports digestion!!
word_tokenize      : ['drinking', '8', 'glasses', 'of', 'water', 'daily', 'supports', 'digestion', '!', '!']
RegexpTokenizer      : ['drinking', 'glasses', 'of', 'water', 'daily', 'supports', 'digestion']
```

Figure 14: Tokenization of the raw text, showing the difference between the two tokenizers.

2.4 Stopword Removal

After tokenization, English stopwords such as *a*, *is*, *of* and *for* were removed because they add little meaning for analysis. Every sentence became shorter; for example sentence 4 went from 7 tokens to 4. Overall, the vocabulary decreased from 77 unique words before stopwords removal to 63 unique words after removal.

```
[7]: stop_words = set(stopwords.words('english'))

def remove_stopwords(tokens):
    result = []
    for word in tokens:
        if word not in stop_words:
            result.append(word)
    return result

df["no_stopwords"] = df["word_tokens"].apply(remove_stopwords)

df["count_before"] = df["word_tokens"].apply(len)
df["count_after"] = df["no_stopwords"].apply(len)

df[["clean_text", "count_before", "count_after"]]
```

```
[7]:
```

	clean_text	count_before	count_after
0	a balanced diet helps maintain healthy body weight	8	7
1	regular workout sessions improve muscle strength love it	8	7
2	drinking glasses of water daily supports digestion	7	6
3	skipping meals can slow down your metabolism	7	4
4	stretching before a workout reduces injury risk important	8	6
5	strength training builds muscle improves bone density	7	7
6	getting hrs of sleep is important for recovery	8	5
7	yoga is a low impact workout improves flexibility	8	6
8	a high sugar diet is linked to health problems	9	6
9	interval training burns calories quickly great results	7	7
10	nutritionists design diet plans based on individual goals	8	7
11	rest days are an important part of any workout schedule	10	6

Figure 15: Token counts before and after stopword removal.

```
[8]: for i in range(len(df)):
      print(i + 1, df["no_stopwords"][i])

1 ['balanced', 'diet', 'helps', 'maintain', 'healthy', 'body', 'weight']
2 ['regular', 'workout', 'sessions', 'improve', 'muscle', 'strength', 'love']
3 ['drinking', 'glasses', 'water', 'daily', 'supports', 'digestion']
4 ['skipping', 'meals', 'slow', 'metabolism']
5 ['stretching', 'workout', 'reduces', 'injury', 'risk', 'important']
6 ['strength', 'training', 'builds', 'muscle', 'improves', 'bone', 'density']
7 ['getting', 'hrs', 'sleep', 'important', 'recovery']
8 ['yoga', 'low', 'impact', 'workout', 'improves', 'flexibility']
9 ['high', 'sugar', 'diet', 'linked', 'health', 'problems']
10 ['interval', 'training', 'burns', 'calories', 'quickly', 'great', 'results']
11 ['nutritionists', 'design', 'diet', 'plans', 'based', 'individual', 'goals']
12 ['rest', 'days', 'important', 'part', 'workout', 'schedule']
```

Figure 16: Tokens remaining in each sentence after stopword removal.

2.5 Stemming and Lemmatization

The remaining words were processed using the **Porter Stemmer** and the **Snowball Stemmer**. Both produced shortened root forms that are often not real words, such as *balanc*, *healthi*, *bodi* and *schedul*. The two stemmers agreed on almost every word, with two exceptions: *quickly* and *hrs*.

```
[9]: porter = PorterStemmer()

def porter_stem(tokens):
    result = []
    for word in tokens:
        result.append(porter.stem(word))
    return result

df["porter"] = df["no_stopwords"].apply(porter_stem)

for i in range(len(df)):
    print(i + 1, df["porter"][i])

1 ['balanc', 'diet', 'help', 'maintain', 'healthi', 'bodi', 'weight']
2 ['regular', 'workout', 'session', 'improv', 'muscl', 'strength', 'love']
3 ['drink', 'glass', 'water', 'daili', 'support', 'digest']
4 ['skip', 'meal', 'slow', 'metabol']
5 ['stretch', 'workout', 'reduc', 'injuri', 'risk', 'import']
6 ['strength', 'train', 'build', 'muscl', 'improv', 'bone', 'densiti']
7 ['get', 'hr', 'sleep', 'import', 'recoveri']
8 ['yoga', 'low', 'impact', 'workout', 'improv', 'flexibl']
9 ['high', 'sugar', 'diet', 'link', 'health', 'problem']
10 ['interv', 'train', 'burn', 'calori', 'quickli', 'great', 'result']
11 ['nutritionist', 'design', 'diet', 'plan', 'base', 'individu', 'goal']
12 ['rest', 'day', 'import', 'part', 'workout', 'schedul']
```

Figure 17: Porter Stemming

```
[10]: snowball = SnowballStemmer('english')

def snowball_stem(tokens):
    result = []
    for word in tokens:
        result.append(snowball.stem(word))
    return result

df["snowball"] = df["no_stopwords"].apply(snowball_stem)

for i in range(len(df)):
    print(i + 1, df["snowball"][i])

1 ['balanc', 'diet', 'help', 'maintain', 'healthi', 'bodi', 'weight']
2 ['regular', 'workout', 'session', 'improv', 'muscl', 'strength', 'love']
3 ['drink', 'glass', 'water', 'daili', 'support', 'digest']
4 ['skip', 'meal', 'slow', 'metabol']
5 ['stretch', 'workout', 'reduc', 'injuri', 'risk', 'import']
6 ['strength', 'train', 'build', 'muscl', 'improv', 'bone', 'densiti']
7 ['get', 'hrs', 'sleep', 'import', 'recoveri']
8 ['yoga', 'low', 'impact', 'workout', 'improv', 'flexibl']
9 ['high', 'sugar', 'diet', 'link', 'health', 'problem']
10 ['interv', 'train', 'burn', 'calori', 'quick', 'great', 'result']
11 ['nutritionist', 'design', 'diet', 'plan', 'base', 'individu', 'goal']
12 ['rest', 'day', 'import', 'part', 'workout', 'schedul']
```

Figure 18: Snowball Stemming

The **WordNet Lemmatizer** was then applied. By default it treats every word as a noun, so verbs like *improves* or *builds* would not change; the notebook therefore tries the word as a verb first and falls back to a noun. The output contains only valid words, for example *balanced* → *balance*, *builds* → *build* and *calories* → *calorie*.

```
[11]: lemmatizer = WordNetLemmatizer()

def lemmatize_word(word):
    lemma = lemmatizer.lemmatize(word, pos='v')    # try as a verb
    if lemma == word:
        lemma = lemmatizer.lemmatize(word, pos='n') # try as a noun
    return lemma

def lemmatize_tokens(tokens):
    result = []
    for word in tokens:
        result.append(lemmatize_word(word))
    return result

df["lemma"] = df["no_stopwords"].apply(lemmatize_tokens)

for i in range(len(df)):
    print(i + 1, df["lemma"][i])

1 ['balance', 'diet', 'help', 'maintain', 'healthy', 'body', 'weight']
2 ['regular', 'workout', 'session', 'improve', 'muscle', 'strength', 'love']
3 ['drink', 'glass', 'water', 'daily', 'support', 'digestion']
4 ['skip', 'meal', 'slow', 'metabolism']
5 ['stretch', 'workout', 'reduce', 'injury', 'risk', 'important']
6 ['strength', 'train', 'build', 'muscle', 'improve', 'bone', 'density']
7 ['get', 'hr', 'sleep', 'important', 'recovery']
8 ['yoga', 'low', 'impact', 'workout', 'improve', 'flexibility']
9 ['high', 'sugar', 'diet', 'link', 'health', 'problem']
10 ['interval', 'train', 'burn', 'calorie', 'quickly', 'great', 'result']
11 ['nutritionist', 'design', 'diet', 'plan', 'base', 'individual', 'goal']
12 ['rest', 'day', 'important', 'part', 'workout', 'schedule']
```

Figure 19: WordNet Lemmatization (verb first, then noun).

2.6 Word Transformation Comparison

The outputs of the three methods were compared for every distinct word after stopwords removal. The comparison shows that stemming produces forms that are not dictionary words, such as *daily* → *daili*, *density* → *densiti* and *balanced* → *balanc*, whereas lemmatization keeps meaningful words such as *daily*, *density* and *balance*.

```
[13]: all_words = []
      for tokens in df["no_stopwords"]:
          for word in tokens:
              if word not in all_words:
                  all_words.append(word)

      rows = []
      for word in all_words:
          p = porter.stem(word)
          s = snowball.stem(word)
          l = lemmatize_word(word)
          if p == s:
              check = "same"
          else:
              check = "DIFFERENT"
          rows.append([word, p, s, l, check])

      compare = pd.DataFrame(rows, columns=["Original Word", "Porter Stem", "Snowball Stem", "Lemma", "Port
      compare
```

Figure 20: Code for comparison of Porter stemming, Snowball stemming and WordNet lemmatization.

[13]:

	Original Word	Porter Stem	Snowball Stem	Lemma	Porter vs Snowball
0	balanced	balanc	balanc	balance	same
1	diet	diet	diet	diet	same
2	helps	help	help	help	same
3	maintain	maintain	maintain	maintain	same
4	healthy	healthi	healthi	healthy	same
5	body	bodi	bodi	body	same
6	weight	weight	weight	weight	same
7	regular	regular	regular	regular	same
8	workout	workout	workout	workout	same
9	sessions	session	session	session	same
10	improve	improv	improv	improve	same
11	muscle	muscl	muscl	muscle	same
12	strength	strength	strength	strength	same
13	love	love	love	love	same
14	drinking	drink	drink	drink	same
15	glasses	glass	glass	glass	same
16	water	water	water	water	same
17	daily	daili	daili	daily	same
18	supports	support	support	support	same
19	digestion	digest	digest	digestion	same
20	skipping	skip	skip	skip	same
21	meals	meal	meal	meal	same
22	slow	slow	slow	slow	same
23	metabolism	metabol	metabol	metabolism	same
24	stretching	stretch	stretch	stretch	same
25	reduces	reduc	reduc	reduce	same
26	injury	injuri	injuri	injury	same
27	risk	risk	risk	risk	same
28	important	import	import	important	same
29	training	train	train	train	same
30	builds	build	build	build	same

Figure 21: Comparison output (part 1).

31	improves	improv	improv	improve	same
32	bone	bone	bone	bone	same
33	density	densiti	densiti	density	same
34	getting	get	get	get	same
35	hrs	hr	hrs	hr	DIFFERENT
36	sleep	sleep	sleep	sleep	same
37	recovery	recoveri	recoveri	recovery	same
38	yoga	yoga	yoga	yoga	same
39	low	low	low	low	same
40	impact	impact	impact	impact	same
41	flexibility	flexibl	flexibl	flexibility	same
42	high	high	high	high	same
43	sugar	sugar	sugar	sugar	same
44	linked	link	link	link	same
45	health	health	health	health	same
46	problems	problem	problem	problem	same
47	interval	interv	interv	interval	same
48	burns	burn	burn	burn	same
49	calories	calori	calori	calorie	same
50	quickly	quickli	quick	quickly	DIFFERENT
51	great	great	great	great	same
52	results	result	result	result	same
53	nutritionists	nutritionist	nutritionist	nutritionist	same
54	design	design	design	design	same
55	plans	plan	plan	plan	same
56	based	base	base	base	same
57	individual	individu	individu	individual	same
58	goals	goal	goal	goal	same
59	rest	rest	rest	rest	same
60	days	day	day	day	same
61	part	part	part	part	same
62	schedule	schedul	schedul	schedule	same

Figure 22: Comparison output (part 2).

Only two words were stemmed differently by Porter and Snowball:

```
[14]: compare[compare["Porter vs Snowball"] == "DIFFERENT"]
```

```
[14]:
```

	Original Word	Porter Stem	Snowball Stem	Lemma	Porter vs Snowball
35	hrs	hr	hrs	hr	DIFFERENT
50	quickly	quickli	quick	quickly	DIFFERENT

Figure 23: Words where the Porter and Snowball stemmers disagree.

2.7 Vocabulary Size Comparison

The vocabulary size was calculated at each major preprocessing stage to see how each technique affected the number of unique words.

Processing Stage	Vocabulary Size
Raw tokens (after cleaning + tokenizing)	77
After stopword removal	63
After Porter stemming	62
After Snowball stemming	62
After lemmatization	62

```
[15]: def vocab_size(column):
      words = set()
      for tokens in df[column]:
          for word in tokens:
              words.add(word)
      return len(words)

      print("Raw tokens (after cleaning + tokenizing):", vocab_size("word_tokens"))
      print("After stopword removal", ": ", vocab_size("no_stopwords"))
      print("After Porter stemming", ": ", vocab_size("porter"))
      print("After Snowball stemming", ": ", vocab_size("snowball"))
      print("After lemmatization", ": ", vocab_size("lemma"))

      Raw tokens (after cleaning + tokenizing): 77
      After stopword removal : 63
      After Porter stemming : 62
      After Snowball stemming : 62
      After lemmatization : 62
```

Figure 24: Vocabulary size at different preprocessing stages.

2.8 Interpretation Questions

1. Why can stemming create invalid dictionary forms while lemmatization returns valid words?

A stemmer follows fixed rules that cut endings like *-ing*, *-es*, *-ly* and *-ity* without checking a dictionary, so the result can be a word that does not exist (*daily* → *daili*, *density* → *densiti*). A lemmatizer looks the word up in the WordNet dictionary and returns its base form, so the output is always a real word (*improves* → *improve*, *calories* → *calorie*).

2. What is the practical difference between `RegexTokenizer` and `word_tokenize`?

`word_tokenize` is a general-purpose tokenizer that splits punctuation into separate tokens and handles contractions (*don't* → *do*, *n't*). `RegexTokenizer` keeps only what matches the given pattern, so with `[a-zA-Z]+` it returns only words and drops numbers and symbols. `word_tokenize` is preferred when punctuation is useful (for example sentence structure or emphasis like "!!!"), and `RegexTokenizer` when only clean words are needed and full control over what counts as a token is required. On the cleaned text of this dataset both produced the same tokens.

3. Did Porter and Snowball produce any disagreement?

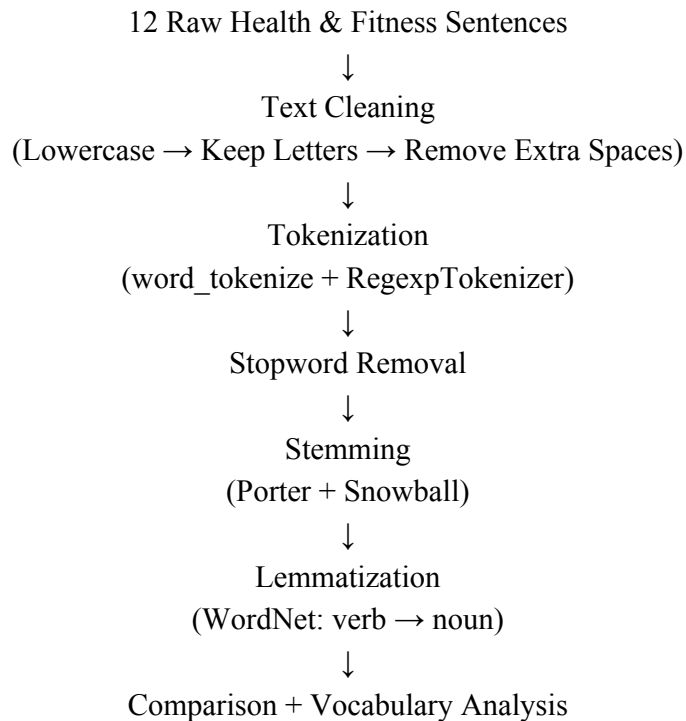
Yes, for two words: **quickly** (Porter → *quickli*, Snowball → *quick*) and **hrs** (Porter → *hr*, Snowball → *hrs*). Snowball (also called Porter2) is an improved version of Porter with updated rules: it removes the *-ly* ending, while Porter only changes the final *y* to *i*, and it does not strip the final *s* from very short words like *hrs*, while Porter does.

2.9 Explanation of Vocabulary Difference

Stopword removal reduced the vocabulary from 77 to 63 words, but stemming and lemmatization reduced it by only one more word (63 → 62). In this dataset most words appear in only one form, so there is little to merge. The single merge comes from *improve* and *improves*, which both become *improv* after stemming and *improve* after lemmatization. Porter and Snowball both end with 62 words even though they disagree on *quickly* and *hrs*, because each of those words still maps to one unique form in both stemmers.

Although the vocabulary sizes are almost equal, the quality of the output is very different: the stemmers produce broken words like *balanc*, *healthi*, *bodi* and *quickli*, while the lemmatizer keeps readable words like *balance*, *healthy*, *body* and *quickly*. For a search engine, stemming is preferred because it is fast and matches many forms of a word together (*improve*, *improves*, *improving* → *improv*). For a classification task or any output that people read, lemmatization is preferred because the words stay meaningful.

2.10 Flowchart



Part 3 - Conclusion

The project demonstrated the use of Word2Vec and text-preprocessing techniques on health and fitness text. Q1 showed how Word2Vec learns word relationships from context: with only 20 sentences, the model found sensible neighbours for *workout* (*workouts*, *muscle*, *improve*) but weaker ones for *diet*, which shows how much these models depend on the amount of training data.

Q2 compared tokenization, stopwords removal, stemming and lemmatization. Stopword removal had the largest effect on vocabulary size (77 → 63), while stemming and lemmatization each removed only one more word. Porter and Snowball disagreed on two words (*quickly* and *hrs*), and lemmatization was the only method that always returned valid dictionary words.

Overall, the project showed that different NLP techniques can produce different representations of the same text, and that understanding each preprocessing step is important when interpreting the final results.